

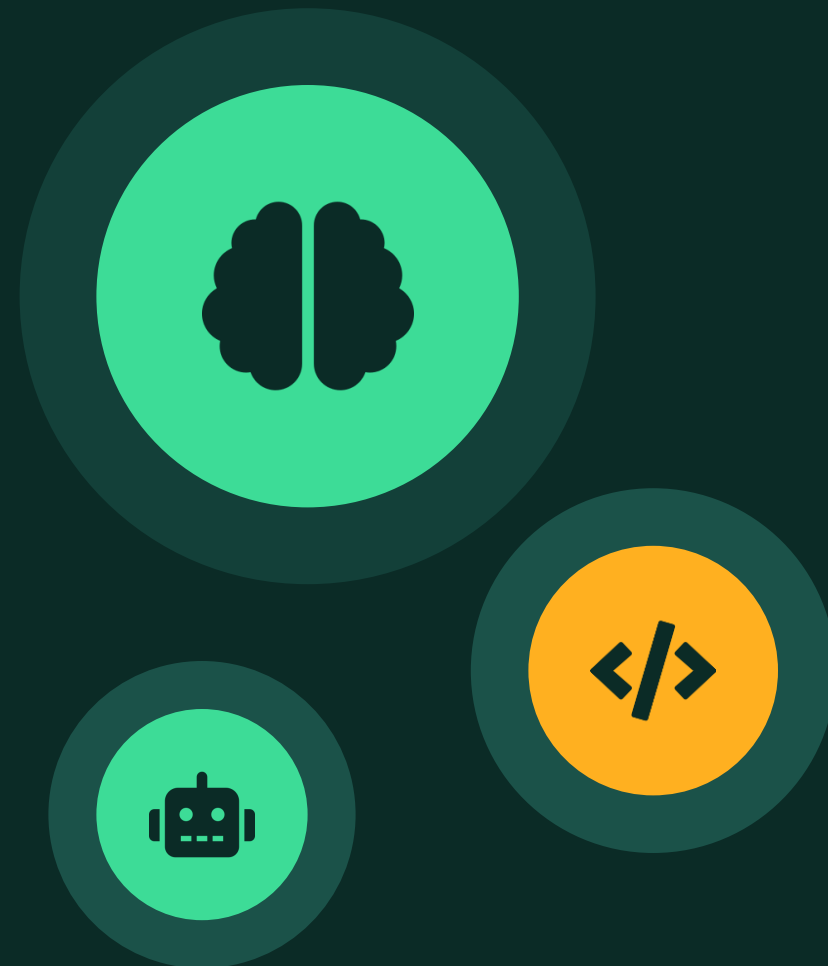
FORMATION INTRA-ENTREPRISE · CGG SERVICES SAS

IA pour les devs

Maîtrisez l'IA pour coder

Jour 1 sur 2 · Développer efficacement avec l'IA

Formateur : Daouda DIOP



Votre formateur



Daouda DIOP

Senior Data Scientist & expert en IA générative



Doctorat en physique – Optique & vision par ordinateur



Chef de projet R&D – Ingénieur R&D, optique/vision par ordinateur



Senior Data Analyst / Data Scientist



Formateur certifié – LLM, RAG, agents IA

PARCOURS PROFESSIONNEL · DANS L'ORDRE

1



Doctorat en physique
Optique / vision par ordinateur

2



Chef de projet
Fibre optique

3



Ingénieur R&D
Optique / vision par ordinateur

4



Ingénieur R&D, couches minces
Ingénieur R&D, optique / vision par ordinateur

5



Chef de projet R&D, optique / vision par ordinateur · Senior Data Analyst / Data Scientist

Depuis 2019

En parallèle



Chef d'entreprise (indépendant)

Ce que vous saurez faire



Utiliser l'IA pour produire, corriger, tester et refactorer du code sur des cas réels



Choisir les outils d'IA les plus adaptés, selon le projet, l'équipe et les contraintes de sécurité



Structurer un workflow de développement qui intègre l'IA de manière fiable et reproductible



Industrialiser l'usage de l'IA, avec documentation, revue de code et automatisation du cycle de développement



Évaluer la qualité, les limites et les risques du code généré, avant de le mettre en production

Comment nous allons travailler ensemble

2 jours

14 heures, en direct et à distance

~50%

pratique

1

projet fil rouge

Certificat

de réussite, envoyé par email

Comment vous êtes évalué

- Travail pratique, partagé avec moi en temps réel (partage d'écran, GitHub...)
- Bilan de formation par email, avec mon évaluation
- Certificat de réussite

Ce que nous attendons de vous

- Une pratique régulière du développement sur un projet réel
- À l'aise avec Git et avec la lecture de code
- Un IDE moderne ou un éditeur de code, utilisé au quotidien
- Votre ordinateur portable

Les deux jours en un coup d'œil

JOUR 1 Développer efficacement avec l'IA

- 1 Comprendre comment l'IA code réellement
- 2 Prompting efficace pour développeurs
- 3 Panorama des outils IA pour développeurs
- 4 Un workflow complet de développement avec l'IA

MISE EN PRATIQUE



Une fonctionnalité complète avec l'IA

JOUR 2 Industrialiser l'usage de l'IA

- 1 Utiliser des agents IA pour des tâches complexes
- 2 Intégrer l'IA dans le workflow d'équipe
- 3 Automatiser le cycle de développement avec l'IA
- 4 Choisir sa stack IA de développement

MISE EN PRATIQUE



Industrialiser le projet avec l'IA

Échauffement : où en êtes-vous aujourd'hui ?



Vos outils

Quels outils IA utilisez-vous ? Chat, assistant d'éditeur, agent... ou aucun pour l'instant.



Votre usage

Pour quelles tâches, et à quelle fréquence ? Code, tests, documentation, debugging ?



Votre meilleure expérience

Quand l'IA vous a-t-elle vraiment fait gagner du temps ?



Votre pire expérience

Qu'est-ce qui a mal tourné, et pourquoi ?



Qu'est-ce que vous voulez retenir, à la fin de ces deux jours ?

JOUR 1 • MODULE 1

Comprendre comment l'IA code réellement

Des LLM aux agents : ce qui se passe réellement quand l'IA écrit
votre code.



Qu'est-ce qu'un LLM ?

Un LLM est un réseau de neurones entraîné à prédire le prochain token.



Grand

Typiquement des milliards de paramètres : des nombres ajustés pendant l'entraînement.



Langage

Entraîné sur d'énormes quantités de texte, incluant beaucoup de code source.



Modèle

Une fonction mathématique : du texte entre, des probabilités sortent.



Ce n'est pas un moteur de recherche, et ce n'est pas une base de données de réponses. Il génère, il ne va pas chercher.

Prédire le prochain token, étape par étape

Ce que le modèle a reçu

```
def add(a, b):  
    return a +
```

Son travail : calculer un score pour chaque token suivant possible

Probabilités du prochain token (illustratives)



Puis un token est choisi, ajouté, et le processus recommence.



Un code plausible est un code probable, pas un code vérifié.

Les tokens : ce qu'un LLM lit et écrit réellement

Découpage illustratif d'un nom de fonction en tokens



Le découpage exact dépend du tokenizer de chaque modèle.



Une règle approximative

En anglais, un token représente environ trois quarts d'un mot. Le code, les mots rares et les autres langues en nécessitent souvent plus.



Tout se compte en tokens

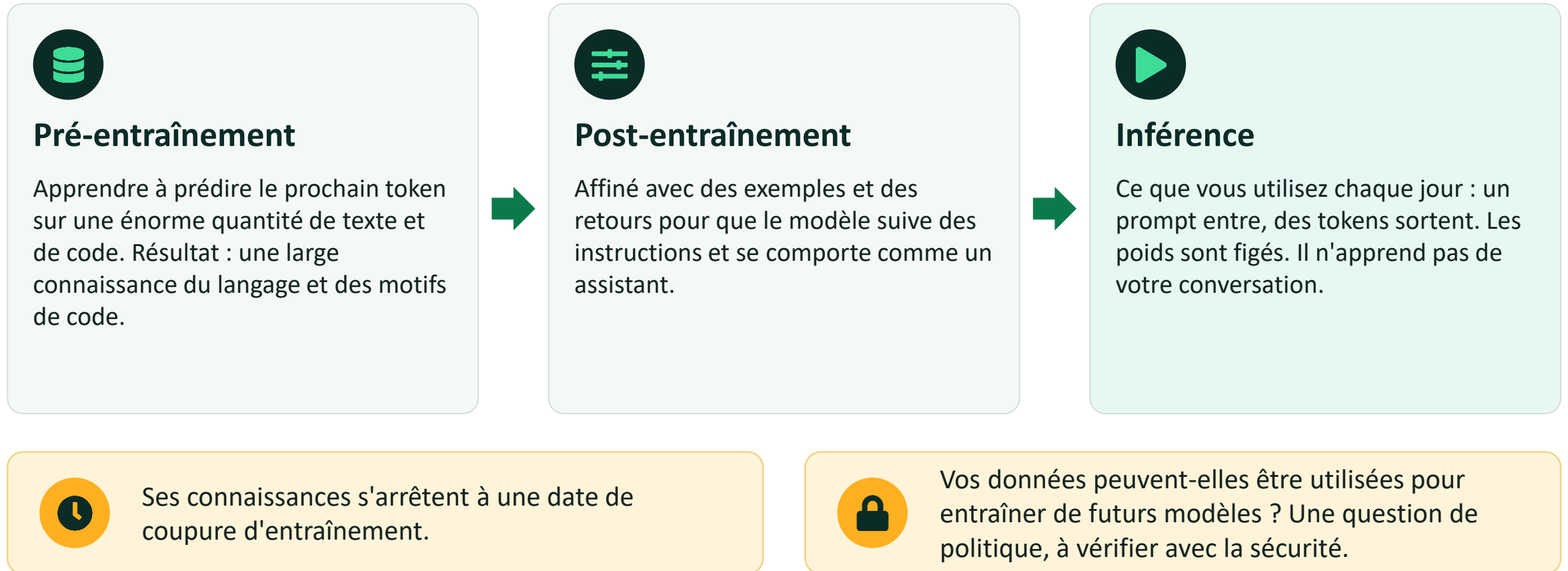
La taille du contexte, la vitesse et, généralement, le prix.



Pas des lettres

Le modèle voit des tokens, pas des caractères : compter des lettres peut être peu fiable.

Comment un LLM est construit : trois étapes



Comment le modèle « lit » votre contexte : l'attention

```
1  user = load_user(user_id)

2  orders = fetch_orders(user)

3  total = compute_total/orders)

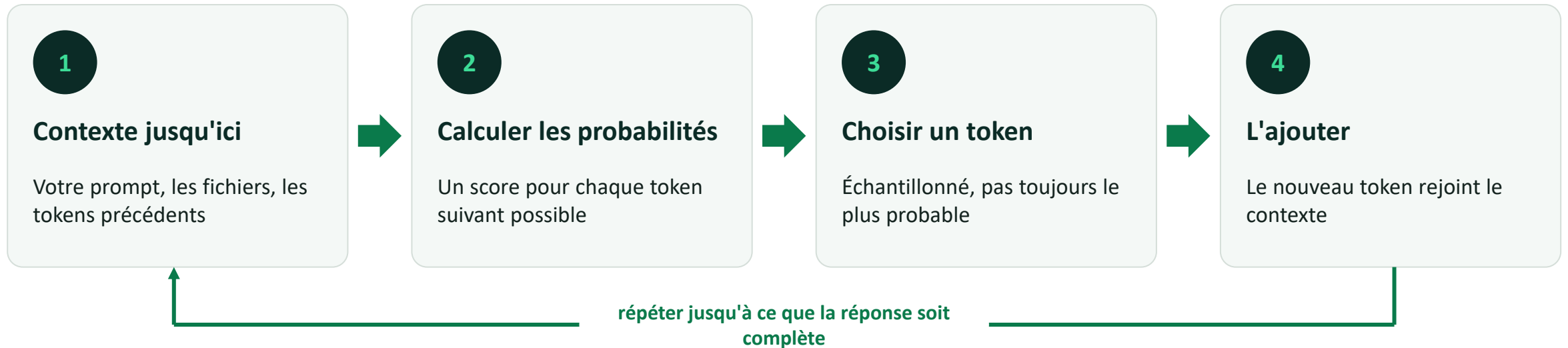
4  return user.
```

« user. » regarde en arrière vers la ligne 1

- **Transformer (2017).** Chaque nouveau token peut regarder en arrière tous les tokens précédents du contexte.
- **Il apprend ce qui compte.** Pour le token courant, quels tokens précédents méritent de l'attention ?
- **Adapté au code.** Un nom défini en haut, utilisé en bas.

Le modèle ne peut prêter attention qu'à ce qui est dans son contexte. Si ce n'est pas là, cela n'existe pas pour le modèle.

Générer du code : une boucle, avec un peu de hasard



Température

Plus basse : plus prévisible. Plus haute : plus variée. Pas toujours accessible dans votre outil.

Même prompt, code différent

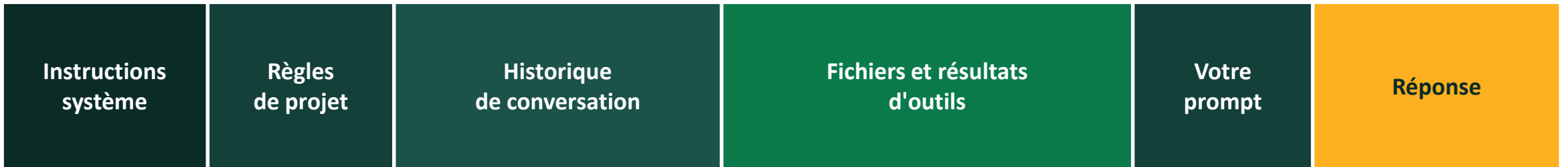
Ne comptez pas sur une sortie identique à chaque fois.

Rendez le processus reproductible

Même modèle, mêmes tests, même revue.

La fenêtre de contexte : le bureau de travail du modèle

Tout ce qui suit doit tenir dans la fenêtre : l'entrée et la sortie ensemble



Une fenêtre partagée

L'entrée et la sortie partagent le même espace. Les réponses longues ont besoin de place.



Quand elle est pleine

L'outil retire ou résume le contenu plus ancien. Des instructions antérieures peuvent être perdues.



Pertinent bat grand

Un contexte long et bruyant a tendance à rendre le modèle moins concentré.

La mémoire : le modèle ne se souvient de rien

Tour 1



Tour 2 : tout est renvoyé



- Sans état. L'application renvoie la conversation à chaque fois.
- Les fonctionnalités de « mémoire » sont du texte enregistré, réinjecté dans le contexte.
- Nouvelle session = page blanche. Sauf si vous rechargez ce qui compte.
- Les connaissances importantes vivent dans des fichiers : documentation, règles de projet, tests.



Si ce n'est pas dans le contexte, cela n'existe pas pour le modèle.

Six limites à garder en tête



Coupure de connaissances

Peut ne pas connaître les versions récentes des bibliothèques ou API, sauf indication contraire.



Aveugle à votre code

Il ne voit que ce que vous, ou votre outil, mettez dans le contexte.



Ne peut rien exécuter

Un simple LLM ne peut ni compiler, ni tester, ni exécuter. Il écrit du texte.



Non déterministe

Le même prompt peut produire des sorties différentes.



Hallucinations

Fluide, confiant, parfois faux.



Contexte fini

Les tâches longues peuvent déborder de la fenêtre.

Hallucinations : confiant, fluide... et faux



Assistant IA

Bien sûr ! Utilisez simplement `requests.get_json(url)`. Cela télécharge la page et retourne directement le JSON parsé.

Cette fonction n'existe pas

Pourquoi cela arrive-t-il ?

- Le modèle génère ce qui est plausible, pas ce qui est vérifié.
- Il n'y a pas d'étape de vérification des faits intégrée.
- Un ton confiant ne dit rien sur la justesse.
- Les trous dans les connaissances sont comblés par des suppositions qui semblent probables.

Exemple écrit pour ce cours, ne provenant pas d'une session réelle.



Dans le code : des fonctions, des paramètres, des options et des packages inventés. Bonne nouvelle : le code est vérifiable.

Huit erreurs fréquentes dans le code généré par IA

1

API inventées

Des fonctions ou des packages qui n'existent pas. Vérifiez chaque dépendance.

2

Usage obsolète

Correct pour une ancienne version de la bibliothèque.

3

Logique plausible mais fausse

Erreurs de décalage d'index, conditions inversées.

4

Cas limites manqués

Valeurs nulles, listes vides, fuseaux horaires, concurrence.

5

Motifs non sécurisés

Validation faible, risques d'injection, secrets dans le code.

6

Ignore les conventions

Style ou architecture qui ne correspond pas à votre projet.

7

Sur-ingénierie

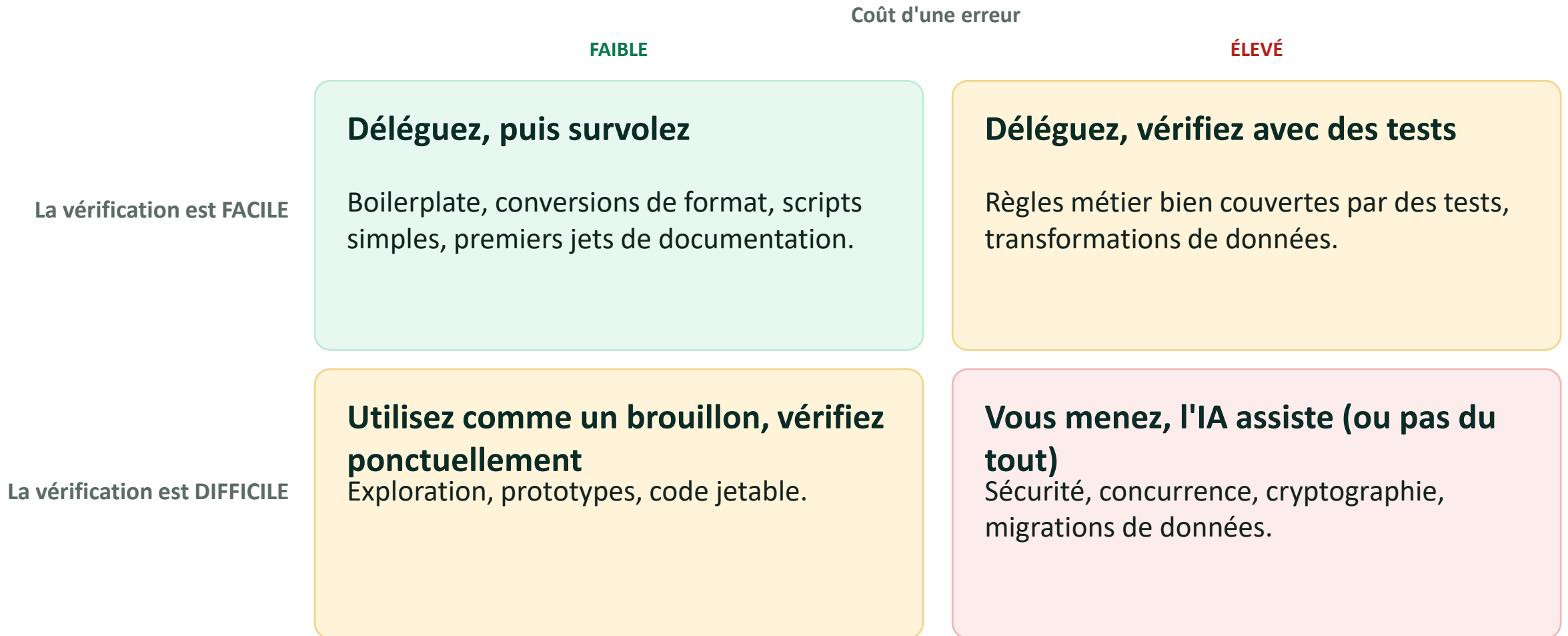
Plus de couches et d'abstractions que nécessaire.

8

Tests vides

Des tests qui passent mais ne prouvent rien.

Calibrer la confiance : combien faut-il vérifier ?



Cinq vérifications avant d'accepter du code généré

1

Est-ce que je comprends chaque ligne ?

2

Est-ce que ça fonctionne, et est-ce que des tests significatifs passent ?

3

Est-ce que toutes les dépendances et API existent vraiment, dans notre version ?

4

Est-ce que ça correspond à nos conventions et à notre architecture ?

5

Est-ce sûr : entrées, secrets, permissions, licences ?



**Vous êtes propriétaire
de ce que vous
commitez.**

Quel que soit qui a écrit le premier
jet.

Quand ne pas utiliser l'IA



Données confidentielles

Ne collez jamais de secrets, de données clients ou de code protégé dans un outil non approuvé.



Vous ne pouvez pas évaluer le résultat

Sans expertise et sans tests : vous ne pouvez pas distinguer ce qui est bon de ce qui est plausible.



Un changement trivial

Plus rapide à faire à la main qu'à expliquer.



Code critique, sans revue d'expert

La logique critique pour la sécurité ou la sûreté a besoin d'un spécialiste.



Besoins flous

Clarifiez d'abord avec les personnes concernées. L'IA construira la mauvaise chose, rapidement.



Apprendre est l'objectif

Pour maîtriser une compétence, débattre un peu d'abord.

Vérification rapide : vrai ou faux ?

1

Un LLM apprend de votre conversation pendant que vous discutez.

2

Une réponse plus confiante a plus de chances d'être correcte.

3

Si quelque chose n'est pas dans le contexte, le modèle ne peut pas y prêter attention.

4

Poser deux fois la même question donne toujours le même code.

5

La fenêtre de contexte inclut à la fois votre entrée et la sortie du modèle.



**Vrai ou faux ?
Répondez dans le
chat.**

JOUR 1 • MODULE 1 • PARTIE B

De LLM à Agent

Ce qui change quand un modèle reçoit des outils et une boucle.



Un LLM seul : du texte entre, du texte sort

Dans le chat



Dans ce dispositif, C'EST VOUS la boucle. Le modèle ne voit aucun fichier, n'exécute rien, et ne voit jamais le résultat.

Donner des mains au modèle : les outils

1

On indique au modèle quels outils existent

Noms, descriptions, paramètres.

2

Il répond avec une requête structurée

« Appelle cet outil avec ces arguments. »

3

Le harnais l'exécute

Puis remet le résultat dans le contexte.

read_file

search_code

edit_file

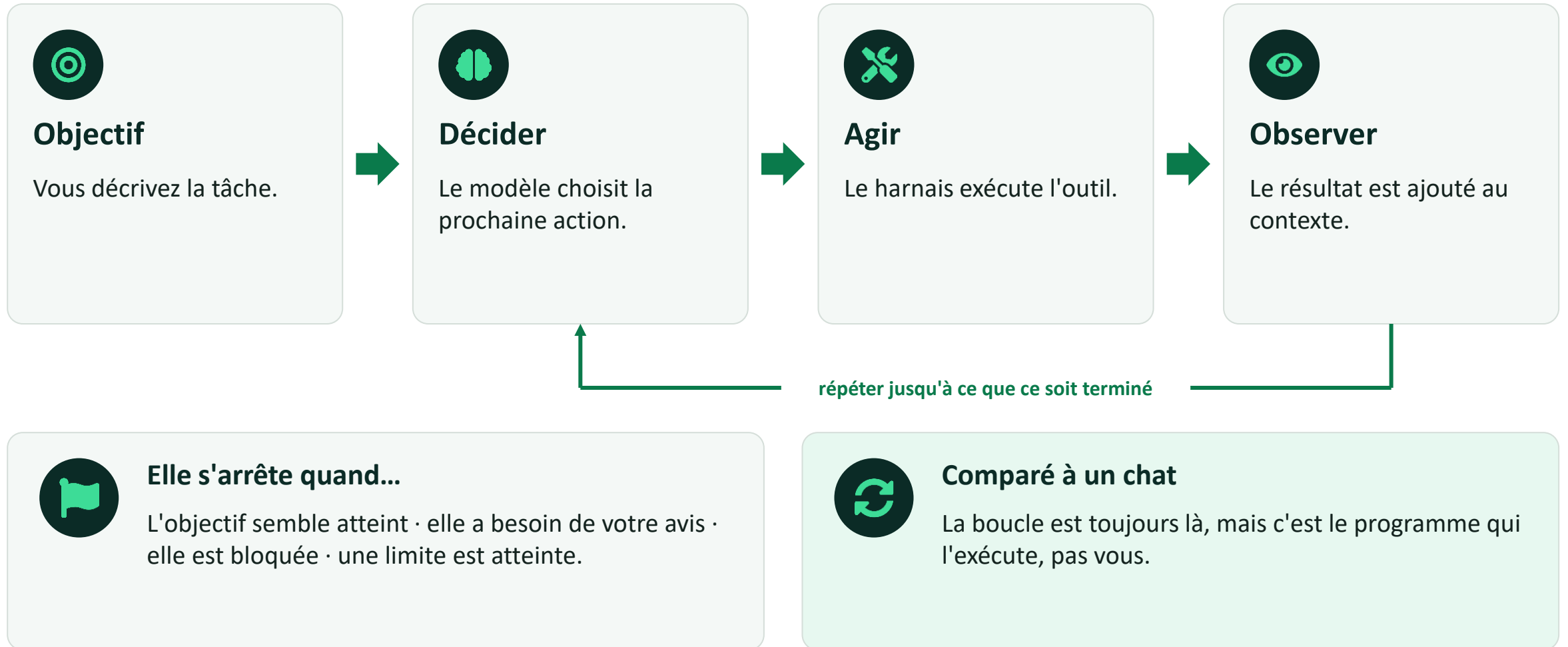
run_command

SORTIE DU MODÈLE

```
// illustrative format
{
  "tool": "read_file",
  "arguments": {
    "path": "src/orders.py"
  }
}
```

**Le modèle n'exécute jamais rien. Il ne fait que demander.**

La boucle de l'agent



LLM vs agent : ce qui change vraiment

	LLM dans un chat	Agent
Ce que c'est	Un modèle qui transforme du texte en texte	Un modèle + des outils + une boucle
Qui pilote	Vous : copier, coller, exécuter, rapporter	L'agent agit et retente ; vous supervisez
Accès à votre code	Seulement ce que vous collez	Cherche et lit les fichiers par lui-même
Peut-il agir ?	Non, il ne fait qu'écrire du texte	Oui : édite des fichiers, exécute des commandes (avec permissions)
Feedback	Vous relayez les erreurs	Lit directement la sortie des tests et les erreurs
Idéal pour	Questions, discussions de conception, extraits de code, explications	Tâches multi-étapes et multi-fichiers
Risque principal	Une mauvaise réponse que vous ne remarquez peut-être pas	De mauvaises actions à grande échelle, des changements non désirés, un coût

Anatomie d'un agent

Agent = Modèle + Outils + Boucle + Contexte + Garde-fous



Modèle

Lit le contexte et décide de la prochaine étape.



Outils

Lire et chercher des fichiers, éditer des fichiers, exécuter des commandes et des tests.



Boucle (harnais)

Le programme qui appelle le modèle, exécute les outils, et renvoie les résultats.



Contexte

Instructions système, règles du projet, fichiers lus, résultats obtenus jusqu'ici.



Garde-fous

Permissions, sandbox, approbations : ce qu'il peut faire seul.



Le harnais compte énormément : deux outils construits sur le même modèle peuvent se comporter différemment.

Une vraie requête, étape par étape

VOUS

Demande : « Corrige le test qui échoue test_parse_date. »

HARNAIS

Construit le contexte : instructions système + règles du projet + liste des outils + votre message. L'envoi au modèle.

MODÈLE

Répond avec une requête d'outil : run_tests.

HARNAIS

Exécute les tests, ajoute la sortie d'échec au contexte, renvoie le tout au modèle.

MODÈLE

Demande read_file sur la fonction, puis edit_file avec un patch. Le harnais exécute chaque requête.

HARNAIS

Relance les tests : ils passent. Le modèle écrit un court résumé.

VOUS

Relisez le diff. Acceptez, ajustez ou rejetez.

Séquence illustrative. Les étapes exactes varient selon l'outil et la tâche.

Trois choses à retenir de cette boucle

1

Le modèle ne produit que du texte

Le harnais fait tout le reste : lire, éditer, exécuter. Les permissions vivent dans le harnais.

2

Chaque étape renvoie un contexte qui grandit

Le coût, la latence et le bruit augmentent à chaque étape. Gardez des tâches petites.

3

L'agent n'est bon que dans la mesure de son feedback

Les tests, les erreurs de compilation et les linters lui permettent de se corriger. Sans cela, il devine.



C'est pourquoi nous gardons les tâches d'agent petites, et donnons toujours à l'agent un moyen de vérifier son propre travail.

Un spectre d'autonomie



Chat

Vous faites les étapes.

L'IA conseille.



IDE assisté par IA

L'IA propose ou applique des éditions dans votre éditeur.

Vous approuvez chaque changement.



Agent autonome

L'IA planifie et exécute des tâches multi-étapes.

Vous relisez le résultat.

Plus de contrôle manuel

Plus de délégation, plus de supervision nécessaire

Beaucoup d'outils couvrent plusieurs points de ce spectre.

Exercice : LLM ou agent ?

- A** Vous collez une expression régulière dans une fenêtre de chat et demandez ce qu'elle fait.
- B** Vous demandez à un outil de trouver où les factures sont validées, d'ajouter une règle, et d'exécuter la suite de tests.
- C** Des suggestions de code apparaissent pendant que vous tapez dans votre éditeur.
- D** Vous collez une stack trace et demandez les causes possibles.
- E** Un outil lit un log de CI qui échoue, édite le code, relance les tests et propose un correctif.

Test : est-ce que quelque chose d'autre que le modèle prend des actions, observe les résultats et boucle ?

Mise en pratique : prise en main et limites des LLM

1.0

Mettre en place le projet pulselab — 10 min

1.1

Vérité sur les LLM : constatez les limites de vos propres yeux — 25 min

APRÈS CE TP

- Gardez votre environnement et votre branche Git prêts
- Notez vos observations dans le journal IA
- Module suivant : prompting efficace pour développeurs

JOUR 1 • MODULE 2

Prompting efficace pour développeurs

Des requêtes vagues à des prompts fiables et réutilisables.



Un prompt est une spécification



Imaginez un collègue très rapide...

...qui a beaucoup lu, mais qui n'a jamais vu votre projet et ne se souvient de rien de la veille.

LE TEST

Si un collègue humain ne pourrait pas faire la tâche à partir de votre seul message, le modèle ne le peut pas non plus.

Soyez précis

Donnez le contexte qu'il ne peut pas deviner

Définissez à quoi ressemble « terminé »

Pourquoi les prompts échouent, et comment les corriger

Symptôme	Cause habituelle	Correction
Réponse générique, inutile	Objectif vague	Indiquez le résultat et la raison
Code qui ne correspond pas au projet	Contexte manquant	Donnez la stack, les versions, les fichiers pertinents, un exemple à suivre
Résultat à moitié fait ou en désordre	Trop de tâches à la fois	Une tâche par prompt ; découpez les grosses tâches
Allers-retours sans fin	Aucune définition de « terminé »	Ajoutez des critères d'acceptation et des tests
Changements non désirés partout	Aucune contrainte	Fixez le périmètre : ce qu'il faut toucher, ce qu'il ne faut pas toucher

L'anatomie d'un bon prompt : six blocs

1

OBJECTIF

Qu'est-ce que je veux, et pourquoi ?
Une tâche claire. Le résultat, pas seulement l'action.

2

CONTEXTE

Qu'est-ce que le modèle a besoin de savoir ?
Stack et versions, code pertinent, erreurs, motifs existants.

3

CONTRAINTES

Que doit-il, ou ne doit-il pas, se passer ?
Périmètre, style, bibliothèques autorisées, fichiers à ne pas toucher.

4

EXEMPLES

À quoi ressemble le bon résultat ?
Exemple d'entrée/sortie, ou une fonction à imiter.

5

FORMAT

Comment la réponse doit-elle être structurée ?
Juste un diff ? Des étapes puis du code ?
Longueur, langue.

6

VÉRIFICATION

Comment saurons-nous que c'est juste ?
Critères d'acceptation, tests, hypothèses, questions ouvertes.

Optionnel : un rôle (« agis en tant que relecteur senior ») peut fixer le ton, mais ne remplace pas le contexte.

De faible à fort : la même requête, réécrite

PROMPT FAIBLE

« Corrige mon code, ça ne marche pas. »

Ce que le modèle doit deviner :

- Langage et version
- Le message d'erreur
- Le comportement attendu
- Ce qui peut être changé
- Ce que « corrigé » signifie

PROMPT FORT

OBJECTIF

Corrige le test qui échoue `test_parse_date_iso` (`tests/test_dates.py`).

CONTEXTE

Python 3.11, pytest. Fonction : `parse_date` dans `utils/dates.py`.

Erreur : `ValueError: time data`

`'2024-03-01T00:00:00Z'` does not match format

`'%Y-%m-%d'`

Attendu : `date(2024, 3, 1)`

CONTRAINTES

Le plus petit changement possible. Bibliothèque standard uniquement. Ne modifie pas le test.

FORMAT

Cause racine en 2-3 phrases, puis un diff unifié.

VÉRIFICATION

Donne la commande pour lancer les tests et liste les effets de bord possibles.

Contexte : donnez au modèle les bons faits



Inclure

- Stack, langage, versions
- Le code pertinent : fonction, signature, types
- Le message d'erreur exact ou la stack trace, copié
- Motifs existants à suivre
- Règles métier qui comptent ici



Laisser de côté

- Déversements de tout le dépôt
- Fichiers non liés et anciennes tentatives échouées
- Secrets, identifiants, données personnelles ou clients
- Suppositions présentées comme des faits



Pertinent bat grand. La plupart des outils permettent de référencer les fichiers directement : utilisez cela plutôt que de tout coller.

D'un prompt personnel à un actif d'équipe



Exemple : un modèle de prompt d'équipe

prompts/bugfix.md

```
# Prompt de correction de bug - v1.2
## Objectif
Corrige : {{bug_summary}}
## Contexte
- Stack : {{language_version}}, {{framework}}
- Fichiers pertinents : {{files}}
- Erreur ou stack trace : {{trace}}
- Attendu vs observé : {{expected}} / {{actual}}
## Contraintes
- Le plus petit changement possible ; aucun fichier non lié
- Ne modifie pas les tests existants sauf mention contraire
- Suis les conventions dans {{conventions_file}}
## Sortie
1. Cause racine (3 phrases maximum)
2. Diff 3. Test ajouté ou mis à jour
## Vérification
- Commande pour lancer les tests
- Risques et effets de bord possibles
```



Variables

{{...}} le rendent réutilisable.



Règles d'équipe, une fois

Les contraintes encodent les conventions pour tout le monde.



Réponses relisibles

Le bloc Sortie façonne ce qui revient.



Versionné

La version dans le titre montre qu'il évolue.

Ne vous répétez pas : les fichiers d'instructions de projet

- Les informations stables vont dans un fichier du dépôt, chargé automatiquement par l'outil.
- Le nom varie selon l'outil : CLAUDE.md, AGENTS.md, ou un fichier de règles spécifique à l'éditeur.
- À y mettre : les commandes de build, de test et de lint ; l'architecture en quelques lignes ; les conventions et les règles « à ne jamais faire » ; des liens vers la documentation.
- Gardez-le court et à jour. Relisez-le comme du code.

EXEMPLE UNIQUEMENT

```
# Guide du projet
## Commandes
- Lancer les tests : pytest -q
- Lint : ruff check .
## Architecture
- api/          couche HTTP
- services/     logique métier
- db/           accès aux données
## Conventions
- Type hints partout ; petites fonctions
- api/ n'appelle jamais db/ directement
## Règles
- Ajoute ou met à jour les tests à chaque
  changement
- Ne commite jamais de secrets
```

Génération de code fiable : six tactiques



1 • Planifier d'abord

Relisez l'approche avant que le moindre code n'existe.



2 • Petites étapes

Une fonction ou un fichier à la fois.
Vérifiez chaque étape.



3 • Montrer un exemple

Pointez vers une fonction existante à imiter.



4 • Le test d'abord

Donnez le comportement attendu sous forme de test, puis générez.



5 • Hypothèses et questions

« Liste tes hypothèses, et pose-moi des questions si quelque chose n'est pas clair. »



6 • Types et signatures

Les interfaces réduisent les suppositions.

Refactoring guidé



EXEMPLE DE PROMPT

```
Refactore process_order() dans services/orders.py.  
Objectif : découpe-la en fonctions plus petites et supprime la duplication.  
Invariant : le comportement doit rester identique ; tous les tests existants passent.  
Contraintes : aucune nouvelle dépendance ; ne change pas les signatures publiques.  
Travaille par petites étapes et montre-moi le diff de chaque étape.
```

Debugging assisté par IA : les preuves, puis la méthode

Donnez les preuves

- Symptôme, message d'erreur, stack trace
- Comportement attendu vs observé
- Une reproduction minimale
- Changements récents
- Ce que vous avez déjà essayé

1

Demandez des hypothèses classées, chacune avec un moyen de la tester

2

Lancez les vérifications (vous, ou l'agent) et réinjectez les résultats

3

Corrigez seulement une fois la cause racine claire

4

Ajoutez un test de non-régression



Demandez la cause avant le correctif. Une hypothèse est une piste, pas un fait : confirmez-la avec une vérification.

Écrire des tests avec l'IA

1

Demandez d'abord un plan de test

Comportements, cas limites, chemins d'erreur.

2

Générez les tests à partir du plan

3

Lisez les assertions

Vérifient-elles quelque chose de réel ?

4

Lancez-les, puis cassez le code exprès

Ils doivent échouer. Sinon, ils ne prouvent rien.

5

Supprimez les tests redondants ou fragiles

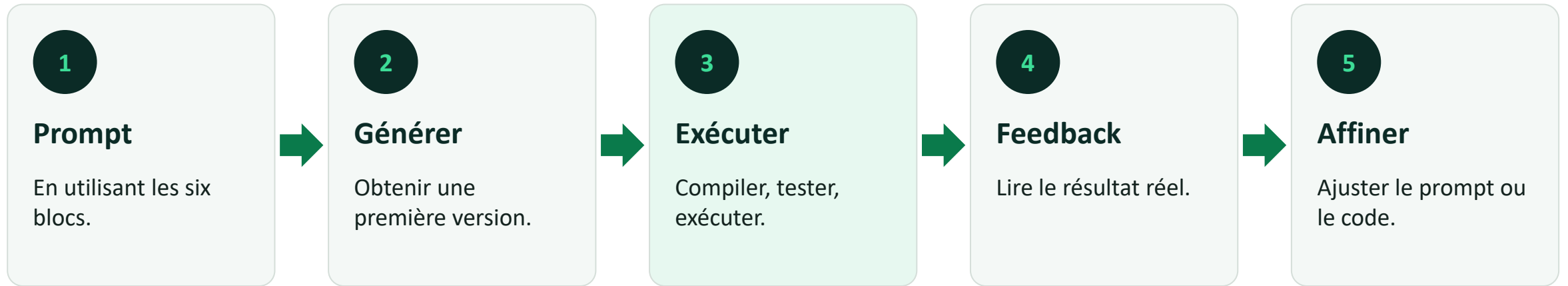


Un test vert n'est pas forcément un bon test.

Méfiez-vous de :

- Tests qui recopient la logique de l'implémentation
- Assertions qui ne peuvent jamais échouer
- Mocking excessif
- Tests adaptés pour faire passer du code faux

Boucles d'itération efficaces



Collez les erreurs exactes

Le message précis, pas une description.

Un changement par tour

Commitez des points de sauvegarde pour pouvoir revenir en arrière.

Sachez quand repartir de zéro

Après 2 à 3 tentatives échouées, repartez de zéro avec un meilleur prompt. Une longue chaîne d'échecs pollue le contexte.

Exercice : réécrire un prompt faible

FAIBLE

« Rends ça plus rapide. »

FAIBLE

« Écris des tests. »

FAIBLE

« Ça ne marche pas, corrige-le. »

1

Choisissez-en un, ou apportez un vrai prompt à vous

2

Réécrivez-le avec les six blocs

3

Lancez les deux versions dans votre outil IA

4

Comparez, et partagez ce qui a changé

Rappel : aucun secret et aucune donnée confidentielle dans vos prompts.

Mise en pratique : du prompt faible au modèle réutilisable

1.2

Atelier de prompting : six blocs et modèle réutilisable — 35 min

APRÈS CE TP

- Gardez votre modèle de prompt sous la main, nous le réutiliserons
- Module suivant : panorama des outils IA pour développeurs

JOUR 1 • MODULE 3

Panorama des outils IA pour développeurs

Chat, assistants IDE et agents autonomes, et comment choisir.



Trois familles d'outils IA

	Chat	IDE assisté par IA	Agent autonome
Où il vit	Conversation dans un navigateur ou une application	À l'intérieur de votre éditeur	Travaille sur votre dépôt, souvent depuis le terminal
Exemples	Assistants de chat généralistes	Cursor, Copilot...	Claude Code, Codex, Opencode...
Le contexte vient de	Ce que vous collez ou téléversez	Fichiers ouverts et espace de travail (selon l'outil)	Explore le dépôt lui-même
Forces	Explorer, apprendre, expliquer, discuter de conception	Éditions et complétions rapides dans le flux ; faciles à relire	Tâches multi-fichiers ; lance les tests et itère
Limites	Contexte manuel ; vous recopiez vous-même les résultats	Le travail long et multi-étapes est plus difficile à piloter	Nécessite supervision et garde-fous ; du temps et un coût

Des caractéristiques typiques, pas des garanties. Les outils évoluent vite, et beaucoup d'outils IDE incluent maintenant des modes agent.

Critères pour choisir ses outils



Type et taille de la tâche

Question, édition dans le flux ou changement multi-fichiers ?



Projet et écosystème

Langage, framework, taille de la codebase.



Confidentialité

La sécurité d'abord : où va votre code ?



Taille et compétences de l'équipe

Que peut utiliser et maintenir l'équipe ?



Intégration

Éditeur, workflow Git, CI/CD.



Coût et licence

Par utilisateur ou par usage ; à quel point est-ce prévisible ?



Contrôle et auditabilité

Pouvez-vous voir et contrôler ce que fait l'outil ?



Flexibilité et lock-in

À quel point est-ce difficile de changer plus tard ?

Questions de sécurité à poser avant d'adopter un outil



Quelles données sortent de la machine, et où sont-elles traitées ?



Sont-elles conservées ? Pendant combien de temps ?



Peuvent-elles être utilisées pour entraîner des modèles ?



Options de déploiement : cloud, privé, modèles locaux ?



Contrôle d'accès, authentification unique, journaux d'audit ?



À quoi l'agent peut-il toucher : fichiers, shell, réseau, secrets ?



Qui possède le code généré ? Qu'en est-il des licences ?



La politique de votre entreprise passe en premier.

Posez ces questions à vos équipes IT et sécurité. Ne présumez pas des réponses.

Quel outil pour quelle tâche ?

La tâche	Un bon point de départ	Pourquoi
Question rapide, explication, brainstorm	Chat	Peu de friction, aucune installation
Écrire et éditer du code dans votre flux	IDE assisté par IA	Des suggestions là où vous travaillez
Fonctionnalité multi-fichiers, refactoring, lot de tests	Agent autonome	Explore, édite, exécute et itère
Code ou données sensibles	Uniquement les outils approuvés par votre entreprise	La politique d'abord
Pas sûr	Commencez petit et lisez chaque diff	Vous construisez votre jugement en vérifiant les résultats

Mise en pratique : IDE assisté vs agent autonome

1.3

Comparer un assistant IDE et un agent autonome — 15 min

APRÈS CE TP

- Notez quand vous avez préféré l'un ou l'autre, et pourquoi
- Module suivant : le workflow complet piloté par l'IA

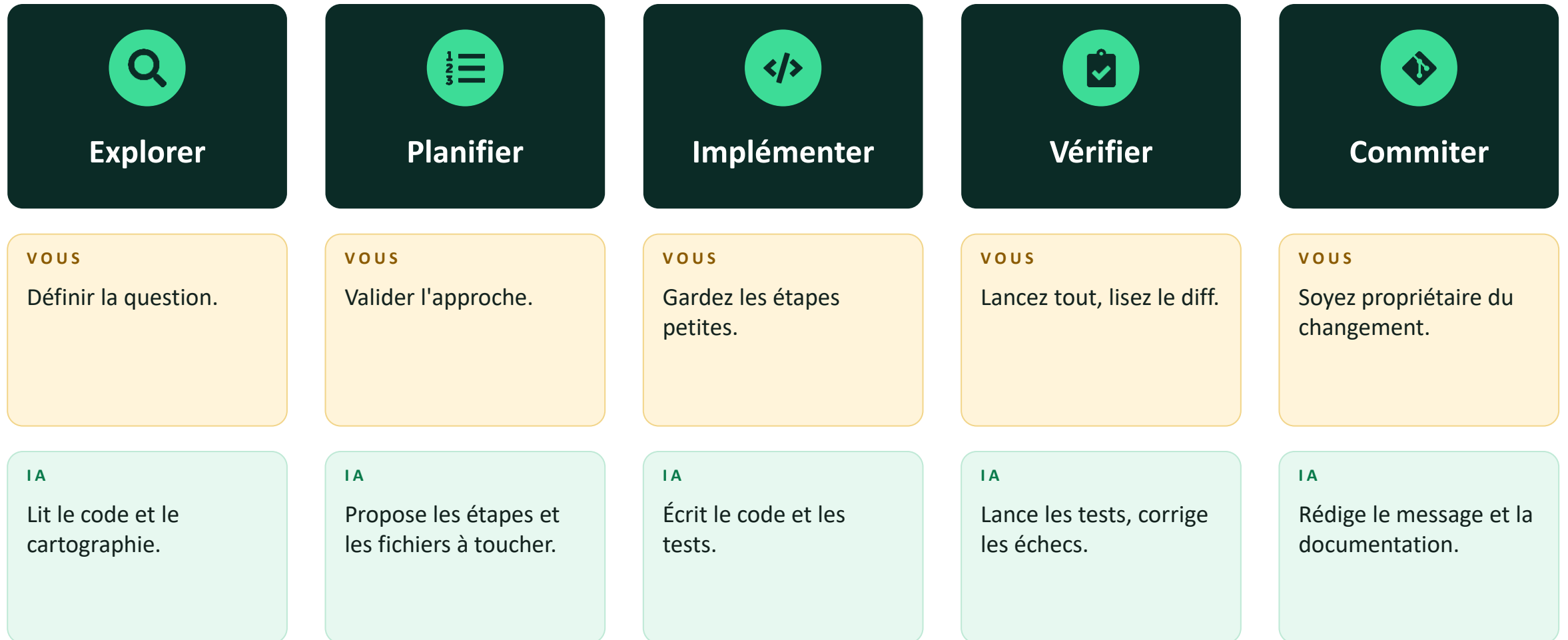
JOUR 1 • MODULE 4

Un workflow complet de développement avec l'IA

Explorer, planifier, implémenter, vérifier, commiter.



La boucle : explorer, planifier, implémenter, vérifier, commiter



Comprendre une codebase existante

IDÉES DE PROMPTS

```
« Donne-moi une carte de ce projet : points  
d'entrée,  
modules principaux et comment les données  
circulent. »
```

```
« Où gère-t-on la validation des factures ?  
Cite les fichiers et les numéros de ligne. »
```

```
« Explique cette fonction et liste ses appelants.  
»
```

```
« Quels tests la couvrent ? »
```



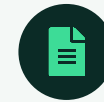
Demandez des preuves

Chemins de fichiers et numéros de ligne.



Ouvrez-les et vérifiez

Un résumé convaincant peut être faux.



Notez vos constats

Un court document dans le dépôt pour l'équipe.



Faible risque, forte valeur : lire du code est l'endroit le plus sûr pour commencer avec l'IA.

Implémenter une fonctionnalité complète

1

Écrivez une mini-spécification

Objectif et critères d'acceptation.

2

Demandez un plan, relisez-le et corrigez-le

Fichiers, étapes, risques. Avant tout code.

3

Implémentez étape par étape

Pas tout d'un coup.

4

Ajoutez ou lancez des tests à chaque étape

5

Relisez le diff, commitez à chaque étape

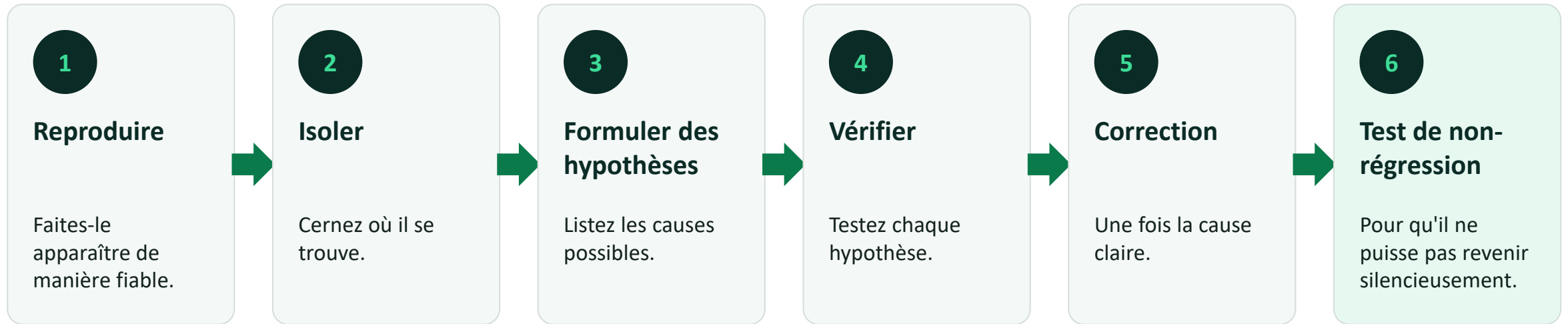
De petits commits relisibles.



Le plan est là où vous avez le plus de levier.

Un mauvais plan produit du mauvais code, très efficacement.

Corriger un bug complexe



Où l'IA aide

Lire les logs et les traces, proposer des hypothèses, suggérer où ajouter du logging, écrire le test de reproduction.



Une astuce pour les agents

Demandez d'abord un test de reproduction. Un test qui échoue est le meilleur rapport de bug : une définition précise et exécutable du succès.

Pense-bête : un motif par tâche

Tâche	Commencez par	Le rôle de l'IA	Votre vérification
Comprendre du code	Une question précise	Cartographie le code et cite les fichiers	Ouvrez les fichiers cités
Nouvelle fonctionnalité	Mini-spec, puis un plan	Implémente par petites étapes	Tests et diff
Bug	Une reproduction qui échoue	Hypothèses, puis un correctif	Test de non-régression
Tests	Une liste de comportements	Génère les cas	Cassez le code exprès
Refactoring	Des tests qui verrouillent le comportement	De petites étapes d'un seul type	Lancez les tests après chaque étape

Mise en pratique : développer une fonctionnalité complète avec l'IA



Explorer une codebase inconnue avec l'IA



Implémenter une fonctionnalité multi-fichiers



Déboguer un bug complexe



Générer et améliorer des tests



Refactoring guidé

RÈGLES DU JEU

- Travaillez sur une branche
- Commitez souvent
- Prenez des notes : ce qui a fonctionné, ce qui n'a pas fonctionné
- Soyez prêts à partager votre écran
- Aucun secret dans les prompts



Debrief : qu'avez-vous observé ?



Où l'IA vous a-t-elle fait gagner du temps ?
Quelles tâches, et pourquoi ?



Où vous a-t-elle induit en erreur ?
Prompt faible, contexte manquant, limite du modèle ?



Assistant IDE ou agent ?
Quand avez-vous préféré l'un ou l'autre ?



Que devrions-nous partager ?
Des prompts ou des règles pour un modèle d'équipe ou un fichier d'instructions.

Le Jour 1 en cinq phrases

- 1 Un LLM prédit des tokens. Plausible n'est pas la même chose que vrai.
- 2 Le contexte, c'est tout ce que le modèle sait de votre tâche. Pertinent bat grand.
- 3 Un agent = modèle + outils + boucle. Le harnais agit, le modèle demande.
- 4 Un bon prompt : objectif, contexte, contraintes, exemples, format, vérification.
- 5 Vérifiez avec les tests et la relecture du diff. Vous êtes propriétaire de ce que vous committez.

Demain : de l'efficacité personnelle à l'industrialisation en équipe.